

UML Narrative: IT Ticketing System with Similar Ticket Detection

System Overview

The IT Ticketing System allows users to submit computer-related problems through support tickets. Each ticket contains information about the user, problem description, category, priority level, and submission date. To reduce duplicate requests, the system includes a similar ticket detection feature that analyzes new ticket descriptions against existing tickets using SQLite Full-Text Search (FTS5) and fuzzy matching techniques.

The purpose of this feature is to help users discover existing issues before submitting a new request and assist IT staff in managing repeated problems efficiently.

Actors

User

The User is responsible for creating and submitting IT support tickets. When entering a new issue description, the User interacts with the system to determine whether similar tickets already exist.

The User can:

- Enter ticket information.
 - Submit a new support request.
 - Review suggested similar tickets.
 - Choose to continue creating a new ticket or use an existing ticket.
-

IT Staff

The IT Staff member manages submitted tickets and uses the system to track, investigate, and resolve computer issues.

The IT Staff can:

- View submitted tickets.
 - Review duplicate or related tickets.
 - Resolve reported problems.
 - Monitor recurring technical issues.
-

Class Descriptions

Ticket Class

The Ticket class represents a support request submitted by a user. It is the primary data model stored in the SQLite database.

Attributes:

- Id: Unique identifier for the ticket.
- DateTime: Date and time the ticket was created.
- UserName: Name of the user submitting the request.
- IssueDescription: Description of the technical problem.
- PriorityLevel: Severity level of the issue.
- Category: Classification of the problem.

The Ticket class is managed by Entity Framework Core and stored in the tickets database table.

TicketDbContext Class

The TicketDbContext class provides communication between the application and the SQLite database using Entity Framework Core.

Responsibilities:

- Store and retrieve Ticket objects.
- Configure database mappings.
- Define relationships and constraints.
- Manage database operations.

The TicketDbContext contains a collection of Ticket objects, representing the one-to-many relationship between the database context and stored tickets.

TicketController Class

The TicketController handles user requests and coordinates the ticket creation process.

Responsibilities:

- Receive new ticket submissions.
- Validate ticket information.
- Request duplicate detection before saving.
- Create new tickets when appropriate.

The controller communicates with the TicketDbContext to store tickets and communicates with the TicketSearchService to check for similar issues.

TicketSearchService Class

The TicketSearchService contains the business logic for identifying possible duplicate tickets.

Responsibilities:

- Receive a new issue description.
- Search existing tickets.
- Compare ticket descriptions.
- Calculate similarity scores.
- Return ranked matching tickets.

The service acts as the connection between database searching and fuzzy similarity calculations.

FTS5SearchEngine Class

The FTS5SearchEngine provides fast text searching using SQLite Full-Text Search.

Responsibilities:

- Index ticket descriptions.
- Search existing ticket descriptions.
- Return possible matching tickets.

FTS5 improves performance by narrowing the number of tickets that require detailed similarity comparison.

FuzzyMatchingService Class

The FuzzyMatchingService calculates how similar two ticket descriptions are.

Responsibilities:

- Compare text values.
- Generate similarity scores.
- Identify possible duplicate tickets.

Possible algorithms include:

- Levenshtein distance.
- Jaro-Winkler similarity.
- Token-based comparison.

The service allows the system to detect minor differences such as:

- "Printer will not print."
- "Printer won't print."
- "Unable to print from printer."

TicketSearchResult Class

The TicketSearchResult class represents the output of a similarity search.

Attributes:

- TicketId: Identifier of the matching ticket.
- IssueDescription: Description of the matching problem.
- SimilarityScore: Numerical similarity rating.
- MatchQuality: Classification of the result.

Examples:

- Exact Match.
 - Very Strong Match.
 - Strong Match.
 - Weak Match.
-

Class Relationships

TicketDbContext and Ticket

The TicketDbContext manages multiple Ticket objects.

Relationship:

TicketDbContext 1 ----- * Ticket

One database context can contain many tickets.

TicketController and TicketDbContext

The TicketController uses TicketDbContext to save new tickets and retrieve existing tickets.

TicketController and TicketSearchService

Before creating a new ticket, the TicketController sends the issue description to the TicketSearchService.

The service determines whether similar tickets already exist.

TicketSearchService and FTS5SearchEngine

The TicketSearchService uses the FTS5SearchEngine to quickly locate possible matching tickets.

TicketSearchService and FuzzyMatchingService

After finding candidate tickets, the TicketSearchService uses the FuzzyMatchingService to calculate similarity scores.

TicketSearchService and TicketSearchResult

The TicketSearchService returns a collection of TicketSearchResult objects containing possible duplicate tickets.

System Workflow

1. The User enters a new IT problem.
 2. The TicketController receives the ticket submission.
 3. The TicketController sends the issue description to the TicketSearchService.
 4. The TicketSearchService searches existing tickets using FTS5.
 5. The FTS5SearchEngine returns possible matching tickets.
 6. The FuzzyMatchingService calculates similarity scores.
 7. The TicketSearchService ranks the results.
 8. Similar tickets are displayed to the User.
 9. The User either:
 - Opens an existing ticket, or
 - Continues with a new ticket submission.
 10. The TicketController saves the new Ticket through Entity Framework Core.
-

Design Purpose

The UML design separates responsibilities between data storage, business logic, and search functionality. The Ticket class represents the data, Entity Framework Core manages persistence, the TicketSearchService handles duplicate detection, and the search components improve the user experience by preventing unnecessary duplicate tickets.

This architecture allows the ticketing system to scale while maintaining clean separation between database operations, application logic, and intelligent search features.